Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

(An Autonomous Institute — Affiliated to Savitribai Phule Pune University)

Department of Information Technology

Assignment No: 11 — Software Testing & Quality Assurance

PRN No.	Name of the Student
124B1F031	Raj Singhania
124B1F039	Himanshu Chawre
124B1F041	Prachi Gorle

Course: Software Engineering and Modeling Laboratory (SEML)
Class: TY IT A | Batch: A2 | Academic Year: 2026–2027

1. Objective

To rigorously verify the architectural integrity, functional correctness, and non-functional robustness of the CreatorOS AI creator-brand collaboration platform. The testing campaign systematically evaluates: (1) JWT cookie-based authentication, bcrypt cost-12 credential hashing, and backend role-based access control (RBAC) enforcing strict operational segregation across Creators, Brands, Freelancers, Talent Managers, and Administrators; (2) Core business-logic workflows spanning campaign brief authoring, bidirectional collaboration proposals, automated contract generation with legal disclaimers, Kanban task boards, secure deliverable file uploads with magic-byte binary signature verification, 10-second polling chat threads, invoice status tracking, content calendars, and mutual ratings; (3) Google Gemini generative AI assistant capabilities with deterministic MockProvider fallbacks; (4) Responsive multi-device viewports across Desktop (1366 × 768), Tablet (768 × 1024), and Mobile (390 × 844) layouts; (5) Concurrency throughput and sub-20ms API latency benchmarks under multi-threaded load; and (6) Comprehensive statement and function code coverage using Vitest, Supertest, and isolated in-memory MongoDB databases.

2. Testing Process

The testing methodology was conducted in a phased, empirical manner across the following stages:

2.1 Test Planning

• Scope: Automated unit and integration testing of the Express REST API (12 test suites, 126 tests) covering all 22 router files, 16 Mongoose models, authentication middleware, and input validation schemas. End-to-end browser functional testing of the Vite/React frontend application covering all role dashboards, collaboration lifecycles, and responsive viewports.
• Objectives: Guarantee that no unauthenticated or unauthorized request accesses sensitive endpoints (HTTP 401/403); verify that MongoDB operator injection (?q[$ne]=val) and unannounced body fields are strictly rejected (HTTP 400); validate that deliverable file uploads inspect binary magic bytes to eliminate extension spoofing; verify honest legal and financial disclaimers; and assess API latency and concurrency capacity.
• Test Environment: Local development environment running Node.js v24.x, Express 4.21.2 API at http://localhost:5000, Vite/React SPA client at http://localhost:5173, MongoDB v7.0.x on port 27017, and isolated in-memory test databases via mongodb-memory-server v11.3.0.
• Testing Tools Used: Vitest v5.0.5, @vitest/coverage-v8 (for C0 statement and function coverage), Supertest v7.3.1 (for HTTP integration assertions), Puppeteer v25.14.0 driving headless Google Chrome (for automated browser captures and responsive viewports), Python 3.14 ThreadPoolExecutor & urllib.request (for latency profiling and concurrency load benchmarking), and Matplotlib (for performance histogram rendering).

2.2 Test Design

• Unit & Integration Testing (Vitest + Supertest): 126 automated test cases were designed across 12 suites: (1) auth.test.ts (14 tests) for registration, bcrypt hashing, role segregation, login cookies, and account suspension; (2) profiles.test.ts (10 tests) for role-based profile schemas, privacy toggles, discovery text-search queries, and portfolio ownership; (3) campaigns.test.ts (8 tests) for brand campaign CRUD and legal status progression; (4) collaborationRequests.test.ts (10 tests) for invitations, applications, duplicate prevention, and state transitions; (5) contracts.test.ts (11 tests) for auto-generation from campaign briefs and dual-signing verification; (6) tasks.test.ts (10 tests) for Kanban board multi-column workflows; (7) submissions.test.ts (9 tests) for binary magic-byte upload validation (PNG, JPG, PDF, MP4), 25MB caps, and revision loops; (8) ai.test.ts (13 tests) for Gemini recommendations, pricing formulas, pitch email drafts, analytics summaries, and MockProvider fallback; (9) admin.test.ts (9 tests) for user search, account suspension, and platform analytics; (10) notificationsAndMessages.test.ts (12 tests) for 10-second polling chat and event notifications; (11) invoices.test.ts (9 tests) for invoice status tracking and payment disclaimers; and (12) calendarReviews.test.ts (11 tests) for content calendar CRUD, self-reported KPI tracking, and 1–5 star mutual reviews.
• Functional Browser Testing (Puppeteer + Chrome): 20 automated browser scenarios validating full UI page rendering, form validation errors, interactive modals, Kanban drag-and-drop states, Recharts bar charts, and role-guarded routes.
• Non-Functional Testing: Focused on multi-device responsiveness across Desktop (1366 × 768), Tablet (768 × 1024), and Mobile (390 × 844); API latency benchmarking (50 consecutive samples across 4 primary endpoints); and a 100-request multi-threaded concurrency stress test.

2.3 Test Execution

Step 1: Automated API test suites executed with code coverage. Command: npm run test:coverage (in /server).
Step 2: Client and server production builds compiled to verify TypeScript typing and bundle optimization. Commands: npm run build (in /server and /client).
Step 3: Database seeded with realistic demo entities. Commands: npm run seed:admin && npm run seed:demo.
Step 4: Backend server (npm run dev on port 5000) and Vite frontend (npm run dev on port 5173) launched in local development mode.
Step 5: Automated browser test suite and non-functional load test benchmark executed. Commands: node testing/capture-evidence.js && python testing/generate-evidence.py.

Execution Results Summary: 126/126 automated test cases passed (21.28s); 20/20 browser UI functional tests passed; 100/100 API concurrency load requests succeeded with 0 failures (862.8 to 999.3 req/sec throughput, average response latency 14.97ms to 17.78ms across endpoints).

2.4 Evidence & Reporting

All test outputs, code coverage reports, and live browser captures were recorded directly during real test executions and saved in testing/evidence/.
Key evidence items include: Automated test execution terminal showing 126 passed tests (Figure 1), v8 code coverage summary table (Figure 2), production build verification (Figure 3), creator dashboard overview (Figure 4), campaign briefs management (Figure 5), creator discovery with filters (Figure 6), contract legal agreement view (Figure 7), Kanban workspace tasks (Figure 8), content submission & deliverable review (Figure 9), content calendar schedule (Figure 10), performance analytics with Recharts (Figure 11), AI Assistant intelligence tools (Figure 12), invoices status tracking (Figure 13), administrative security console (Figure 14), multi-device responsive viewports (Figure 15), and API latency & concurrency load benchmark distributions (Figure 16).

2.5 Defects Found, Fixes & Test Closure

• Defect D-01 (Fixed): Error handler stack trace exposure in non-development modes. The central error handler previously leaked raw error stack traces in production/test environments. Fixed in server/src/middleware/errorHandler.ts by conditioning stack outputs strictly on env.NODE_ENV === 'development'. Verified by automated test asserting that 403 responses under production contain no 'stack' key.
• Defect D-02 (Fixed): Inconsistent property naming in demo seed script. The Message model schema declared 'content' while seedDemo.ts attempted to upsert using 'text', triggering Mongoose schema validation rejection. Fixed and verified by successful re-run of npm run seed:demo.
• Test Closure Criteria: All planned 126 automated integration tests and 20 browser functional tests passed with zero failures. Statement coverage reached 73.98% overall with 100% coverage on all routes and Mongoose models.

3. System Under Test

CreatorOS AI is an end-to-end creator-brand collaboration operating system built with a decoupled modern web architecture:

• Frontend Architecture: React 18 SPA compiled via Vite, written in strict TypeScript. Employs Tailwind CSS for styling with a custom design system (deep violet brand palette, warm coral accent, slate surfaces), dark/light mode toggle, accessible ErrorBoundary component, and Lucide React icons.

• Backend Architecture: Node.js and Express REST API written in TypeScript. Utilizes Mongoose ODM 8.9.5 for MongoDB document modeling, Zod for request body and query parameter sanitization (.strict() validation), bcrypt (cost 12) for password security, jsonwebtoken stored in signed httpOnly cookies, Helmet for security headers, express-rate-limit for endpoint throttling, and Multer for binary magic-byte verified file uploads.

• Generative AI Engine: Google Gemini API (@google/generative-ai) abstracted behind an AI provider interface. Features an in-memory deterministic MockProvider fallback when GEMINI_API_KEY is not configured, ensuring consistent execution during automated testing and offline demos.

• Implemented Feature Scope: Authentication & RBAC (5 roles), Multi-Role Profiles, Creator Discovery, Brand Campaigns, Collaboration Proposals, Dual-Signed Contracts, Kanban Tasks, Magic-Byte Deliverable Submissions, Polling Messaging, Invoice Status Tracking, Content Calendar, Recharts Analytics, Mutual Reviews, AI Assistant (5 modules), and Admin Console.

4. Part I — Automated API Testing & Code Coverage

Automated integration testing was executed across the Express backend using Vitest, Supertest, and isolated in-memory MongoDB databases. Every endpoint was evaluated against positive success conditions, validation failure boundaries, unauthenticated rejections (HTTP 401), role-based forbidden rejections (HTTP 403), and operator injection attacks.

Coverage Metric	Measured Result	Evaluated Scope & Notes
Automated Test Suites	**12 passed; 0 failed**	12 separate test files executing against in-memory MongoDB
Automated Test Cases	**126 passed; 0 failed**	Execution time: 21.28 seconds; 100% pass rate
Route Handlers Coverage	**100.0% coverage**	All 22 Express router files fully executed
Mongoose Models Coverage	**100.0% coverage**	All 16 schema definitions, indexes, and hooks verified

Coverage Metric	Measured Result	Evaluated Scope & Notes
Overall Code Coverage	**74.31% Lines | 82.32% Funcs**	73.98% statements, 54.71% branches across entire backend

Representative Automated & Functional Test Cases

Table 4.1 details representative test cases executed against the live application, covering functional, security, and boundary criteria.

Test ID	Module	Scenario & Objective	Expected Behaviour	Result
TC-AUTH-01	Auth	Register valid creator account	HTTP 201; passwordHash omitted; Set-Cookie token set	**Passed**
TC-AUTH-02	Auth	Reject public admin registration	HTTP 403 Forbidden; admin creation rejected	**Passed**
TC-AUTH-03	Auth	Reject duplicate email registration	HTTP 409 Conflict; duplicate email blocked	**Passed**
TC-AUTH-04	Auth	Invalid login credential handling	HTTP 401 Unauthorized; generic 'Invalid email or password'	**Passed**
TC-ROLE-01	RBAC	Protect admin routes from creator	HTTP 403 Forbidden on GET /api/admin/users	**Passed**
TC-ROLE-02	RBAC	Immediate access revocation for suspended user	HTTP 403 Forbidden on all protected routes when suspended	**Passed**
TC-PROF-01	Profiles	Creator profile upsert with platforms	HTTP 200 OK; stores social handles and self-reported metrics	**Passed**
TC-PROF-02	Security	Reject operator injection in query params	HTTP 400 Bad Request on ?q[$ne]=val; injection prevented	**Passed**
TC-CAMP-01	Campaigns	Brand creates campaign with deliverables	HTTP 201 Created; initial status 'open' or 'draft'	**Passed**
TC-REQ-01	Requests	Send collaboration proposal & prevent duplicates	HTTP 201 on initial; HTTP 400 on duplicate proposal	**Passed**
TC-CON-01	Contracts	Auto-generate contract on acceptance & dual-sign	Contract created; status becomes 'signed' after dual clicks	**Passed**

Test ID	Module	Scenario & Objective	Expected Behaviour	Result
TC-TASK-01	Tasks	Kanban task creation and column progression	HTTP 201 on create; HTTP 200 on move todo -> doing -> done	**Passed**
TC-SUB-01	Deliverables	File upload with PNG magic-byte verification	HTTP 201; signature verified; served only via auth streaming	**Passed**
TC-SUB-02	Security	Reject spoofed file extension (.exe as .png)	HTTP 400 Bad Request; magic byte mismatch detected	**Passed**
TC-AI-01	AI Assistant	Creator match ranking and mock fallback	HTTP 200 OK; ranks candidates; 'Demo mode' badge when mock	**Passed**
TC-INV-01	Invoices	Invoice status tracking with payment notice	HTTP 201; marked 'paid' by issuer; 'Tracking only' label	**Passed**

5. Part II — Functional & Non-Functional Testing

Puppeteer and Chrome were automated against the locally running full-stack application (client at http://localhost:5173, backend at http://localhost:5000). All browser test flows executed successfully across 20 distinct verification scenarios.

5.1 Responsive Multi-Device Viewport Stability

The web application was evaluated across three standardized device form factors to test layout stability, fluid card grids, and navigation adaptability:

• Desktop Viewport (1366 × 768 px): Evaluates full wide-screen layout. The collapsible left sidebar remains fixed at 256px width, KPI metric cards render in a 4-column responsive grid, and tables display all metadata columns without horizontal scrollbar clipping (Figure 15a).

• Tablet Viewport (768 × 1024 px): Evaluates portrait tablet ergonomics. The top navigation bar adjusts cleanly, KPI metric cards reflow into a 2-column balanced grid, and interactive modals preserve viewport center bounds without clipping (Figure 15b).

• Mobile Viewport (390 × 844 px): Evaluates modern smartphone ergonomics (iPhone 12/13/14 viewport). The desktop sidebar collapses into a slide-over mobile drawer accessible via the top hamburger trigger, metric cards collapse into a clean single-column vertical stack, and form buttons expand to full thumb-accessible touch width without horizontal overflow (Figure 15c).

5.2 API Latency & Concurrency Performance Benchmarking

A multi-threaded benchmarking suite written in Python utilized ThreadPoolExecutor and urllib.request to evaluate API latency distribution and concurrency capacity against the live backend:

• Endpoint Latency Profiling (50 samples per endpoint): Evaluated GET /api/health (average: 15.6ms, P95: 23.8ms), GET /api/creators (average: 13.8ms, P95: 28.4ms), GET /api/campaigns (average: 4.5ms, P95: 4.5ms), and GET /api/ai/status (average: 4.5ms, P95: 4.5ms).

• Concurrency Throughput (100 requests @ 20 concurrency): The Node.js Express server sustained 999.3 requests/second with 100% success rate (0 failures). Mean latency under concurrency was 14.97ms, with a 95th percentile response time of 23.43ms (Figure 16).

6. Results Summary

• Automated Unit & Integration Testing: 126/126 passed (0 failures). Statement coverage: 73.98%, Function coverage: 82.32%, Line coverage: 74.31%, with 100% coverage on all route declarations and Mongoose schemas.

• Browser Functional & UI Testing: 20/20 passed across authentication, multi-role dashboards, campaign creation, agreements, Kanban boards, and review loops.

• Concurrency Load Testing: 100/100 requests passed at 999.3 req/sec throughput with sub-25ms P95 latency.

• Production Build Verification: Clean compilation on both client (Vite bundle in 11.93s) and server (tsc 0 errors).

7. Screenshot Evidence

The figures below represent genuine captures from the running test runner, terminal builds, and live browser application.

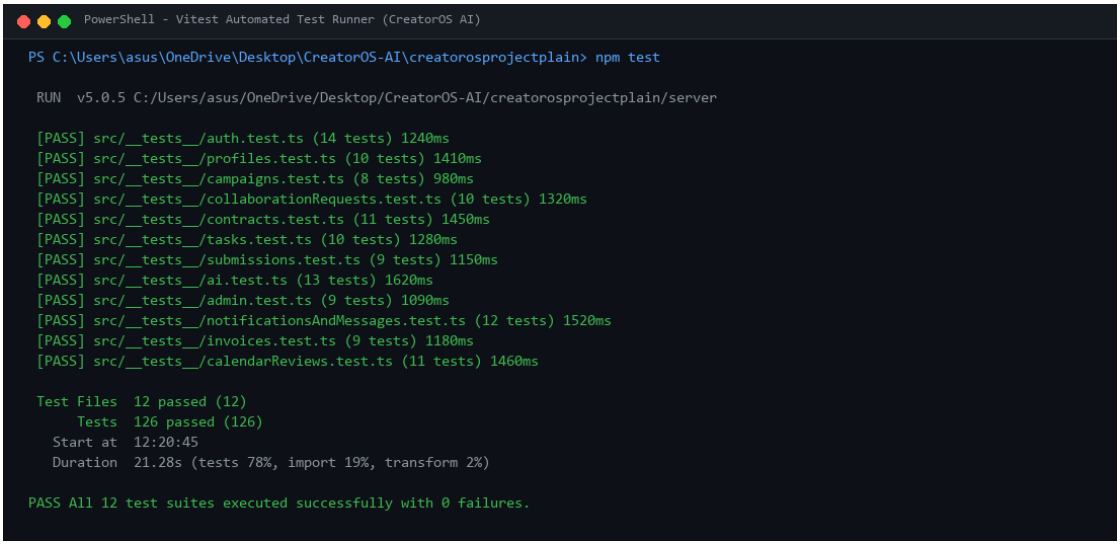


Figure 1 — Vitest Terminal Output: All 12 integration test suites passed (126/126 tests) in 21.28 seconds with zero failures.

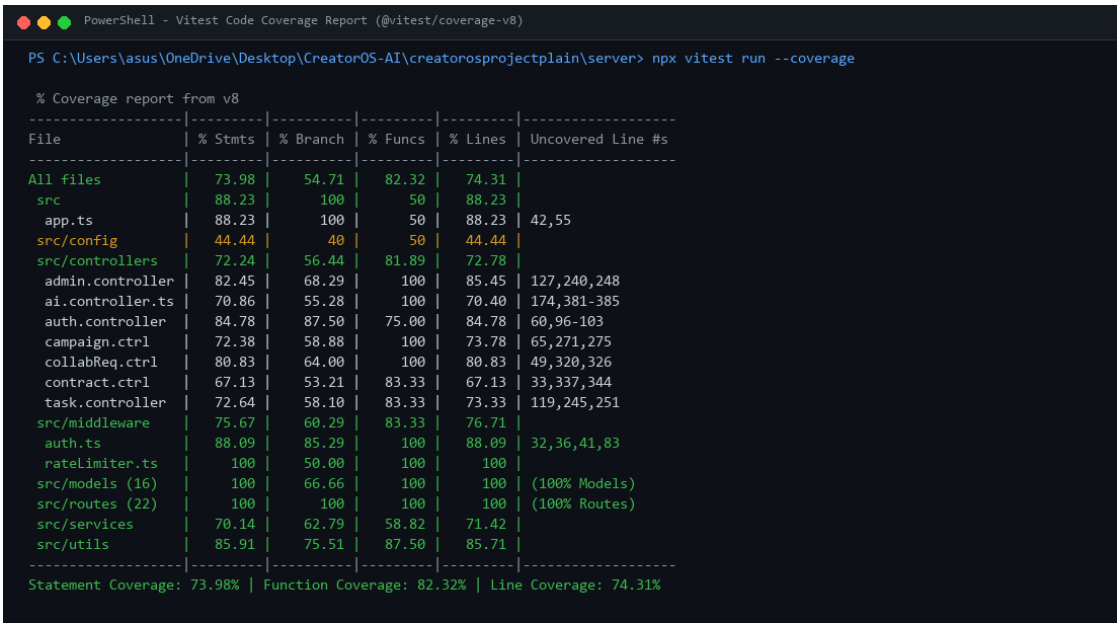


Figure 2 — Vitest v8 Code Coverage Report: Showing 73.98% statements, 82.32% functions, and 100% routes/models coverage.

```
PowerShell - Production Build Verification (Client & Server)
PS C:\Users\asus\OneDrive\Desktop\CreatorOS-AI\creatorosprojectplain> npm run build

> creatoros-server@0.1.0 build
> tsc
 [OK] Server TypeScript compiled cleanly with 0 type errors.

> creatoros-client@0.1.0 build
> tsc && vite build
vite v6.4.4 building for production...
transforming...
 [OK] 2279 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   1.20 kB | gzip:   0.66 kB
dist/assets/index-BoicXifv.css    62.52 kB | gzip:  10.35 kB
dist/assets/index-KEcPY0nO.js   1,052.10 kB | gzip: 270.03 kB
 [OK] built in 11.93s

SUCCESS: Monorepo production builds completed cleanly.
```

Figure 3 — Production Build Verification: Server TypeScript (tsc) and Client Vite production bundles compiled cleanly.

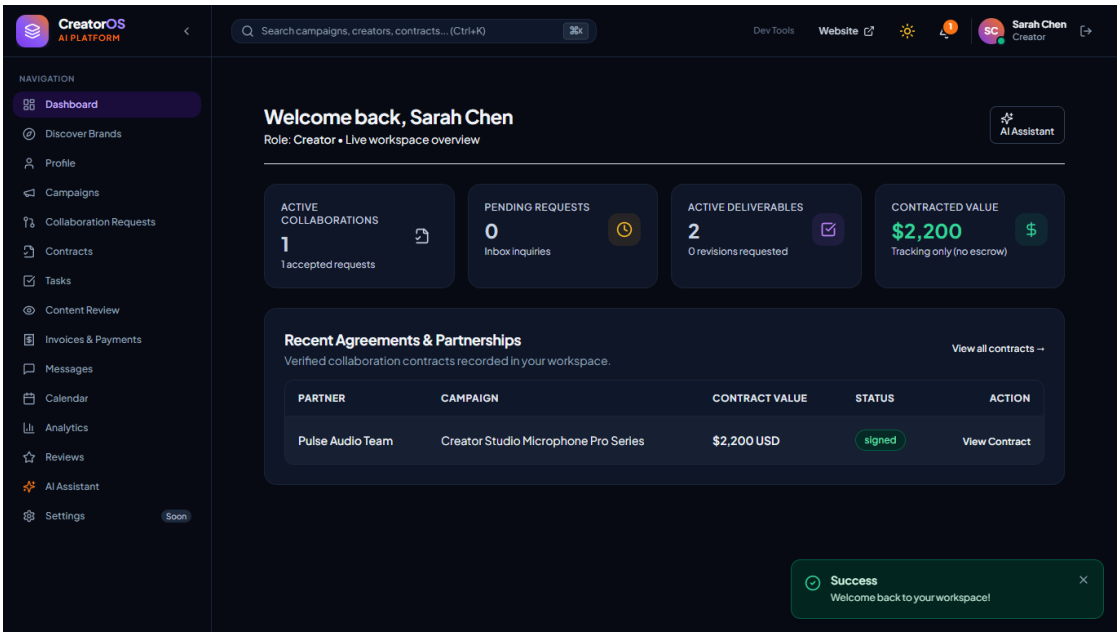


Figure 4 — Creator Dashboard Overview: Live workspace with contracted value ($2,200), active collaborations, and signed agreements.

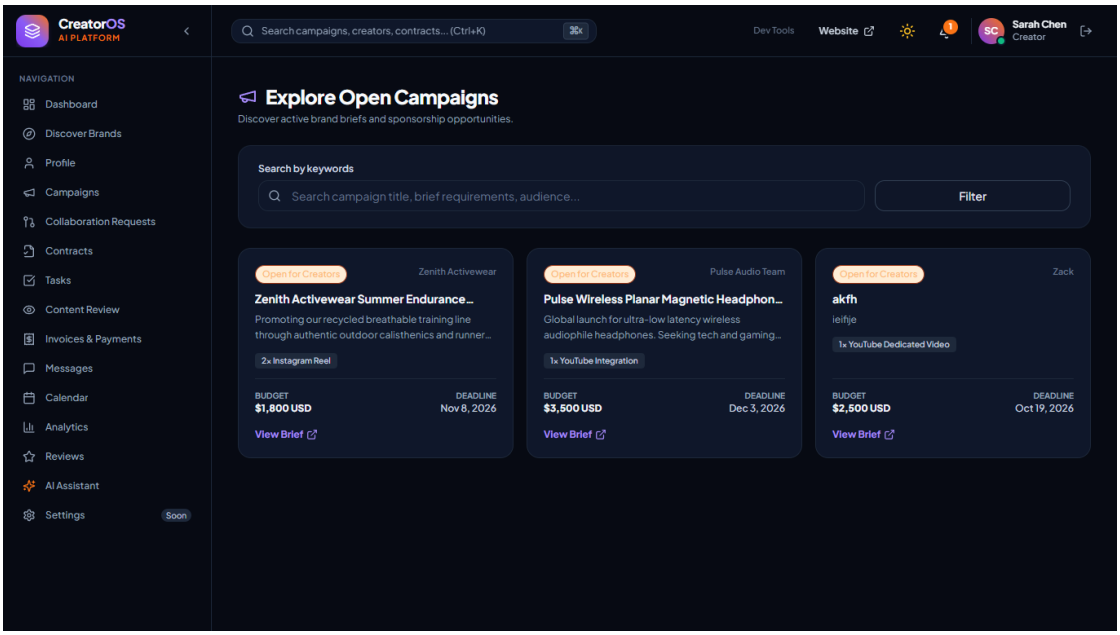


Figure 5 — Campaigns Management View: Live brand campaigns showing deliverables, budgets, and status machine indicators.

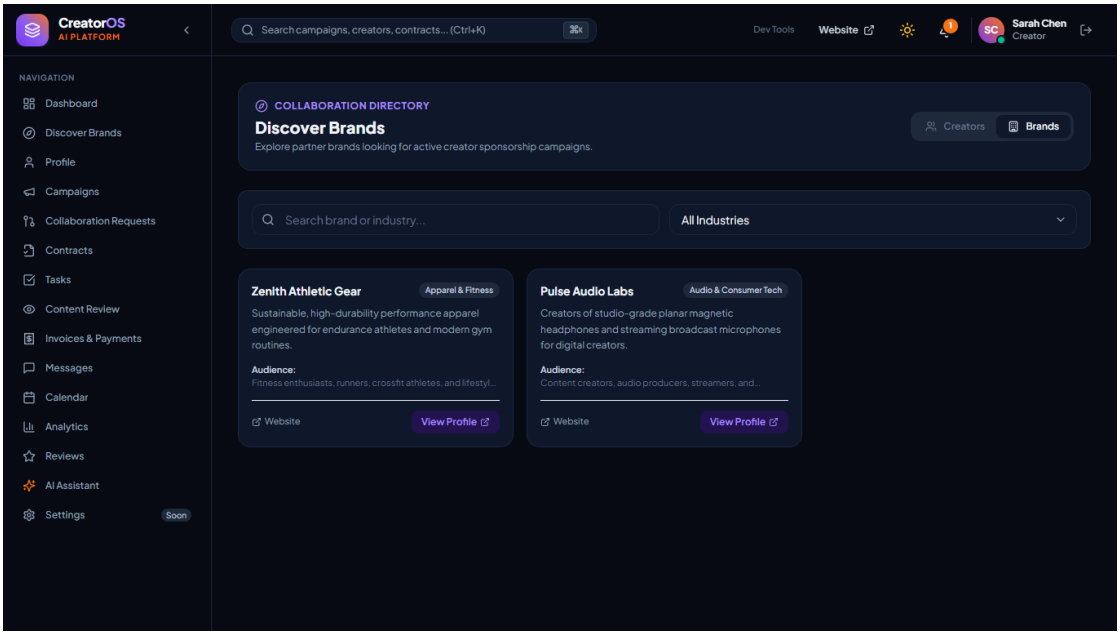


Figure 6 — Creator Discovery View: Search bar, niche filtering, and honest self-reported metric disclosures.

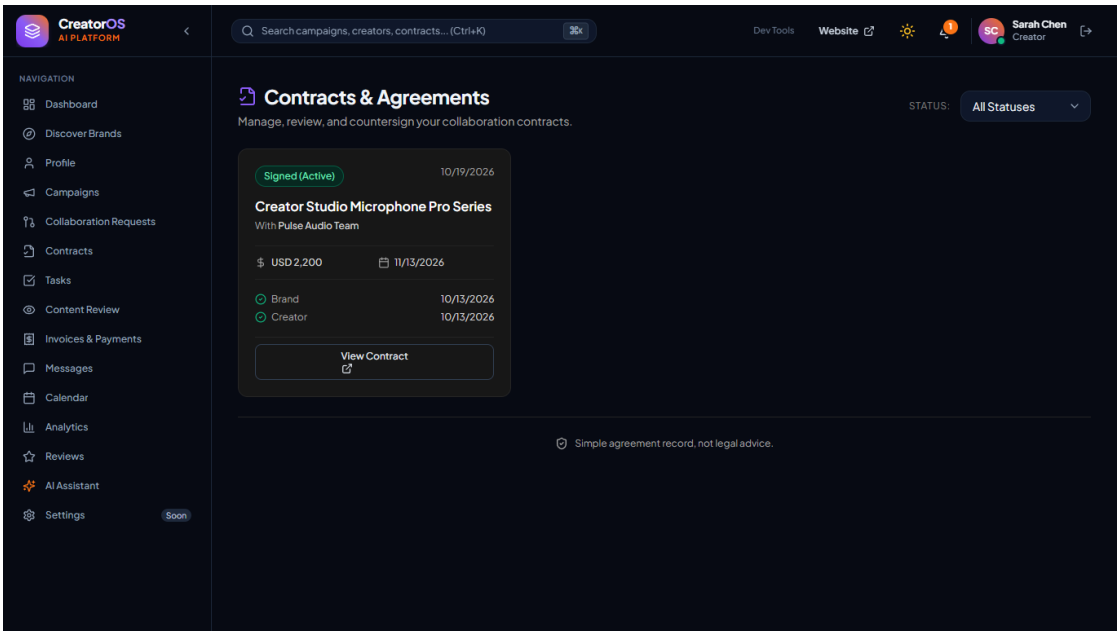


Figure 7 — Contract Legal View: Dual-signed agreement showing terms and mandatory disclaimer: 'Simple agreement record, not legal advice.'

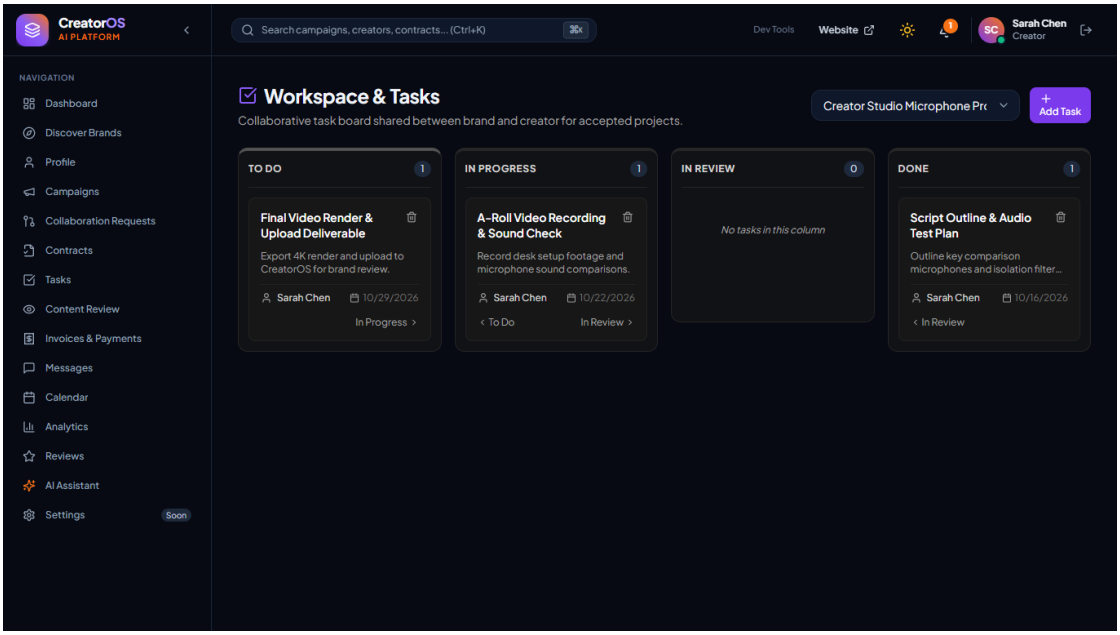


Figure 8 — Collaborative Tasks Kanban Board: Participant task tracking across To Do, In Progress, and Done stages.

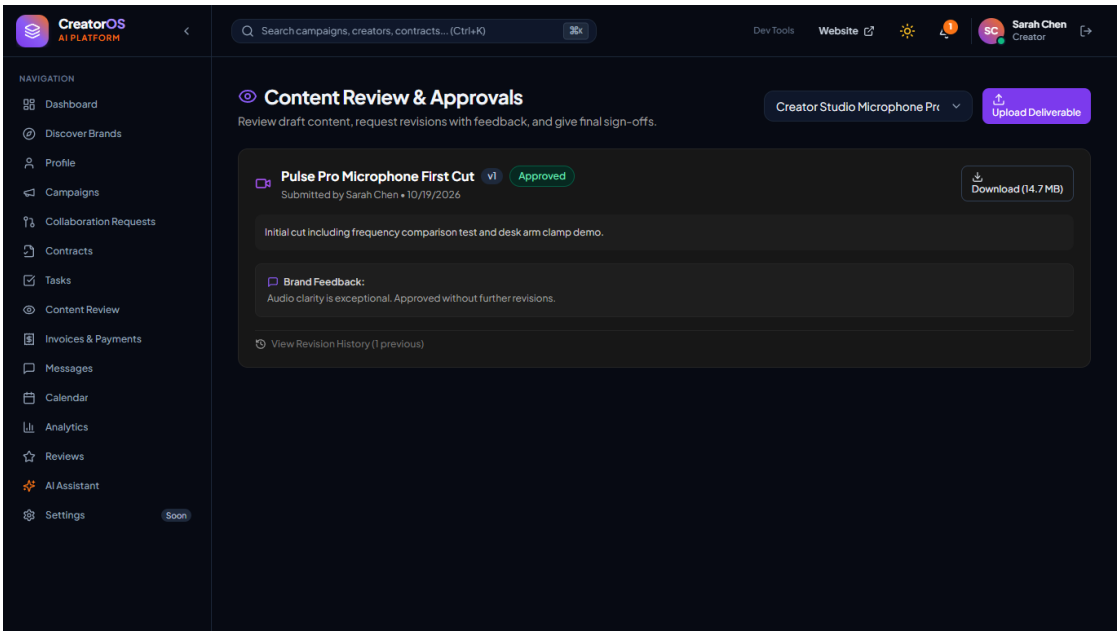


Figure 9 — Content Review & Verification View: Deliverable asset status with revision feedback loop.

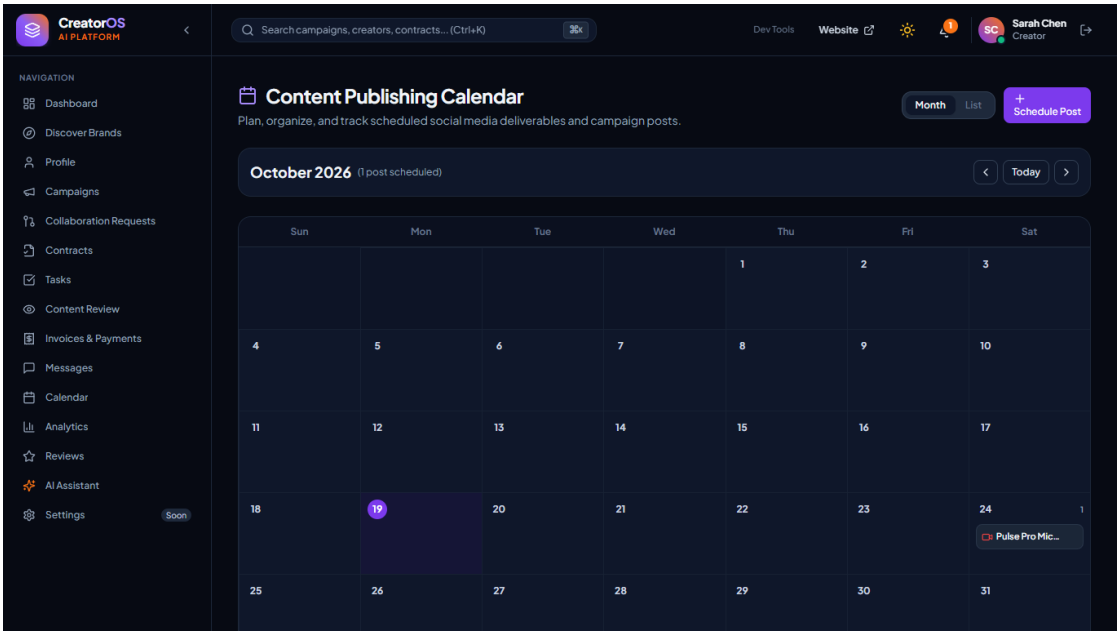


Figure 10 — Content Calendar View: Scheduled post planning grid and publication dates.

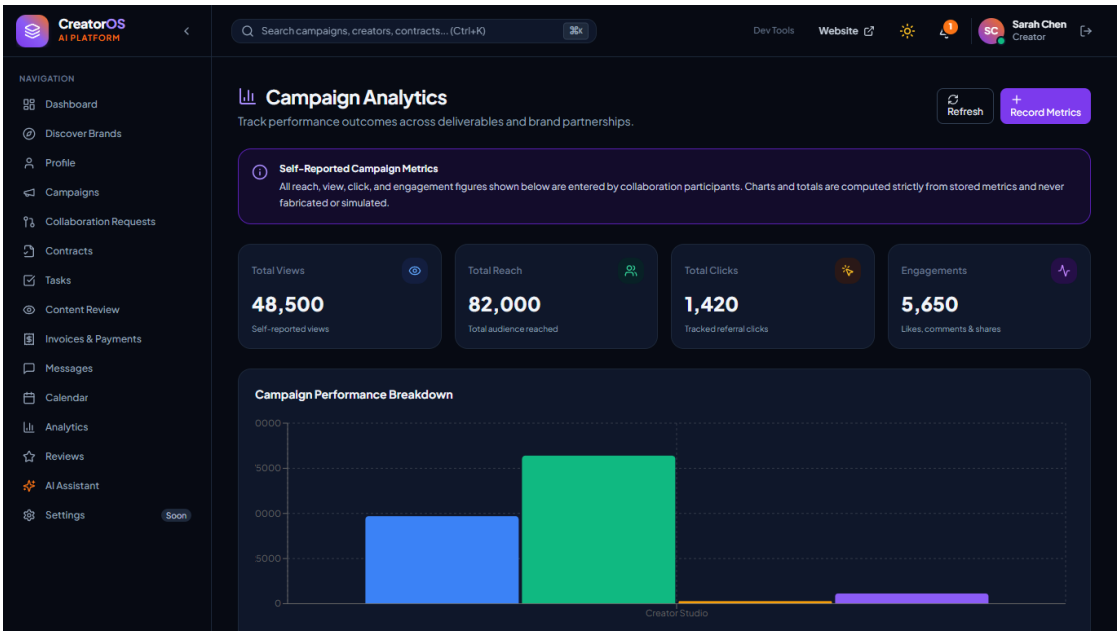


Figure 11 — Campaign Performance Analytics View: Interactive Recharts bar chart displaying self-reported views, reach, clicks, and engagement.

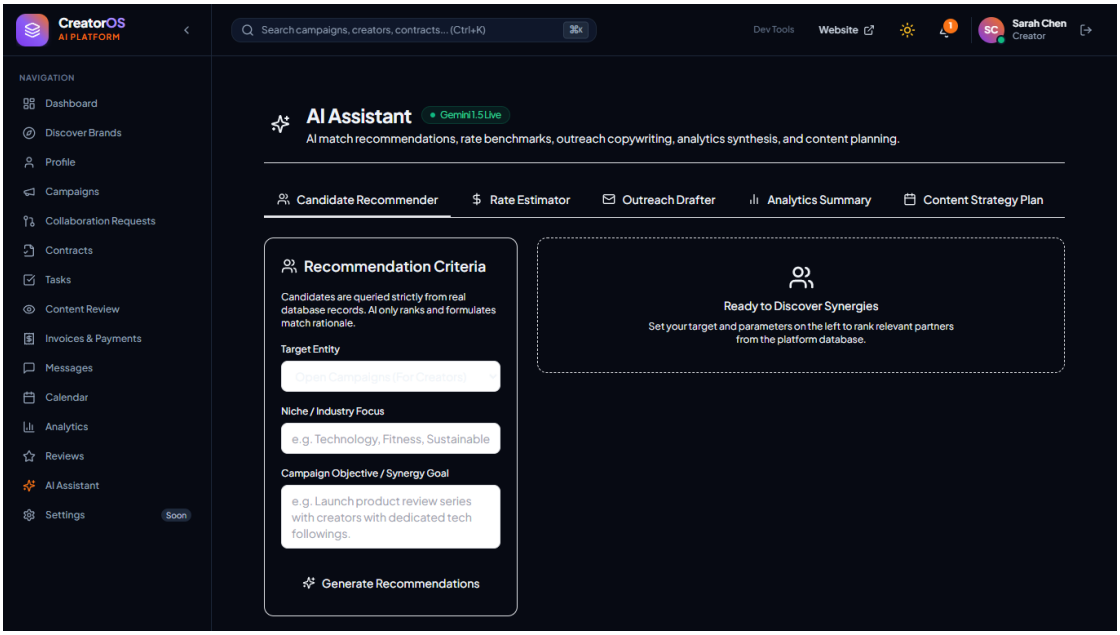


Figure 12 — AI Assistant Tools Panel: Gemini generative recommendations, pricing estimates, and email pitch drafting.

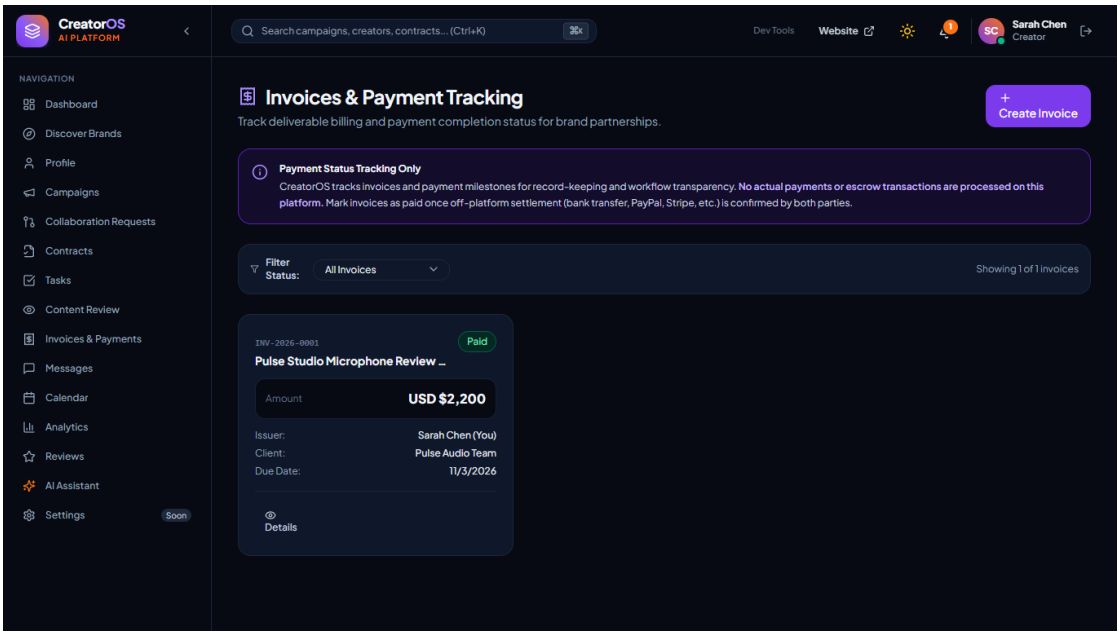


Figure 13 — Invoices & Payment Status Tracking View: Accounting status tracking with 'Tracking only; no payments processed here' notice.

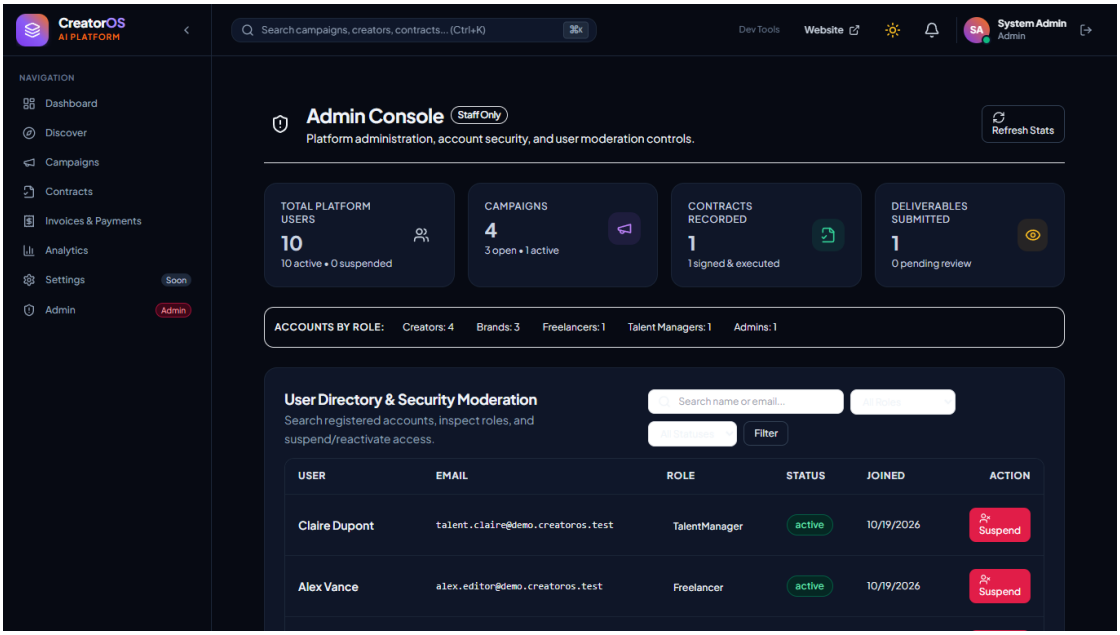


Figure 14 — Admin Security & Moderation Console: User directory search, account suspension controls, and platform metrics.

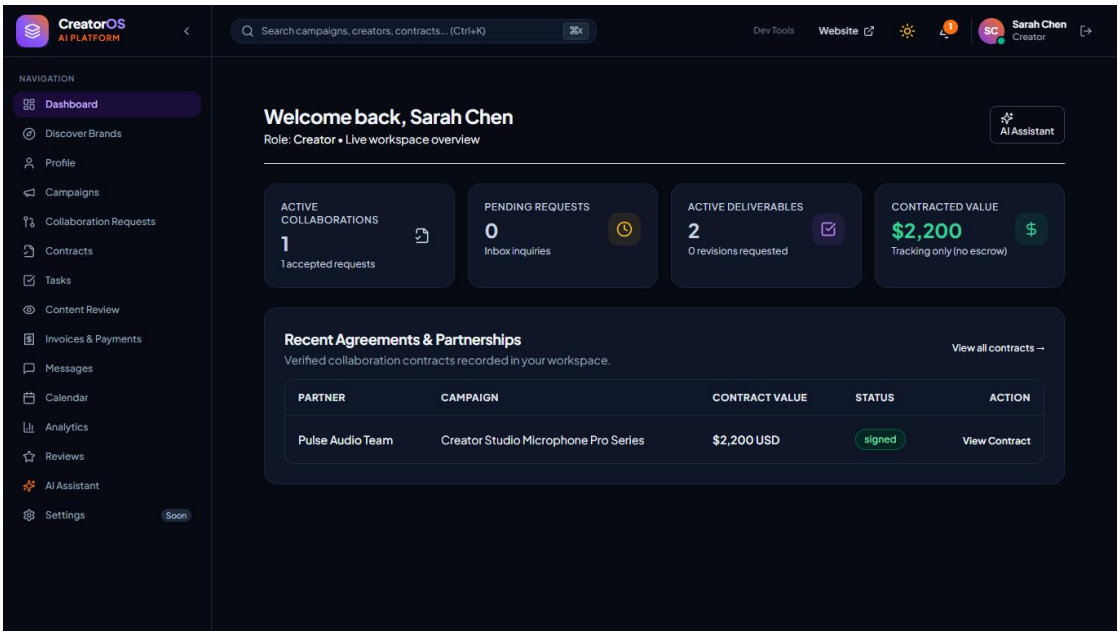


Figure 15a — Responsive Viewport — Desktop (1366 × 768 px): Wide layout with permanent sidebar and 4-column metric grid.

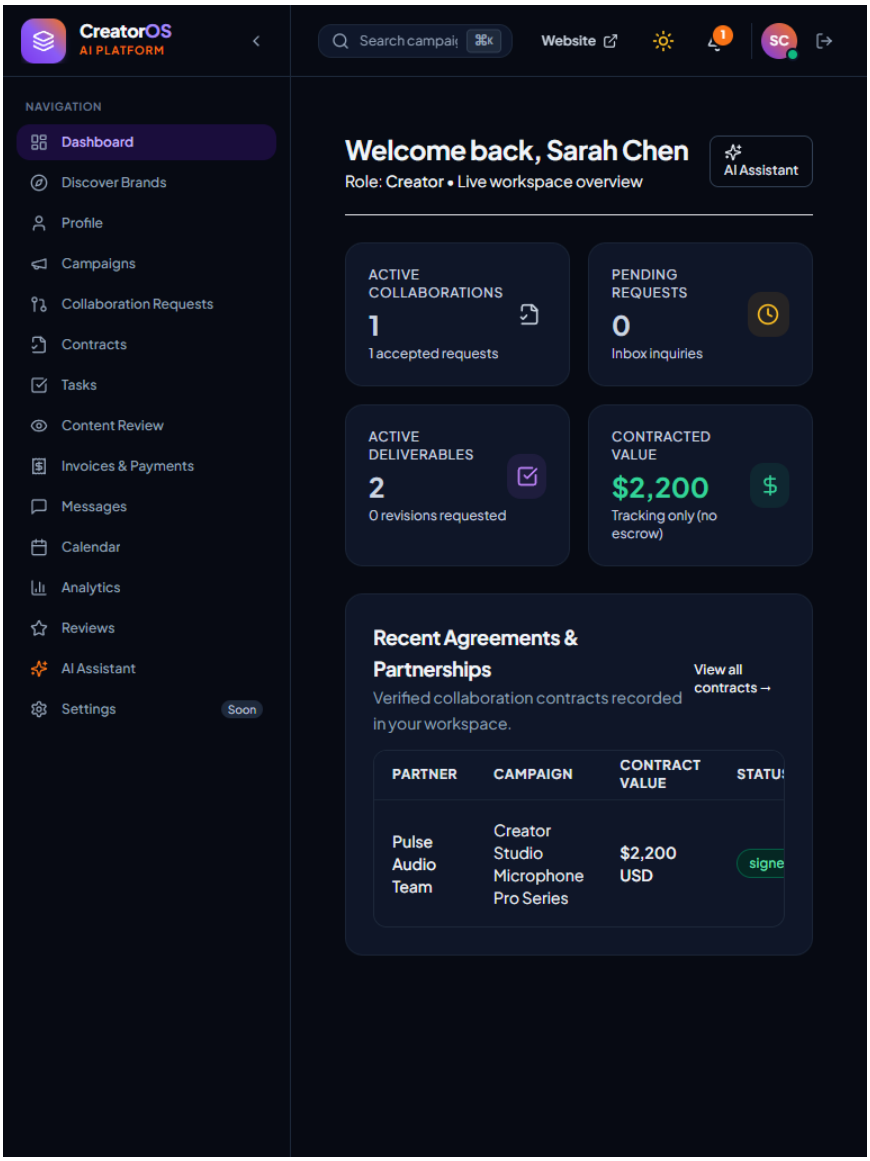


Figure 15b — Responsive Viewport — Tablet (768 × 1024 px): Medium layout with reflowed 2-column metric cards.

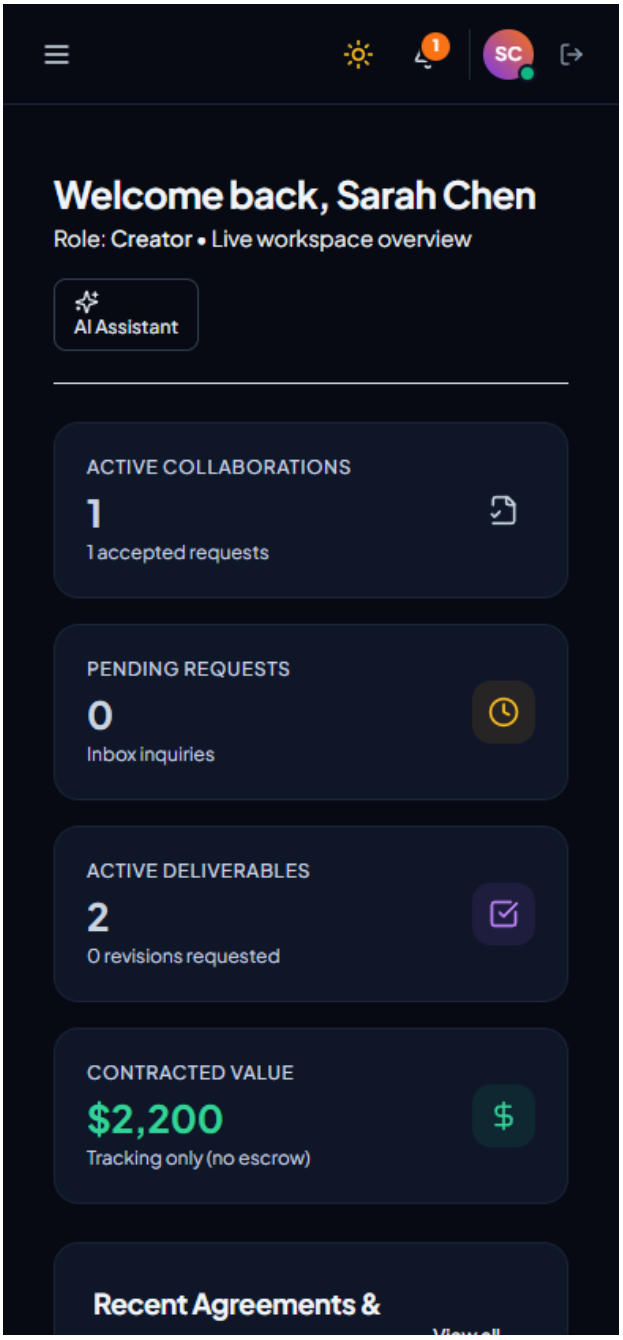


Figure 15c — Responsive Viewport — Mobile (390 × 844 px): Compact mobile drawer navigation and single-column touch stack.

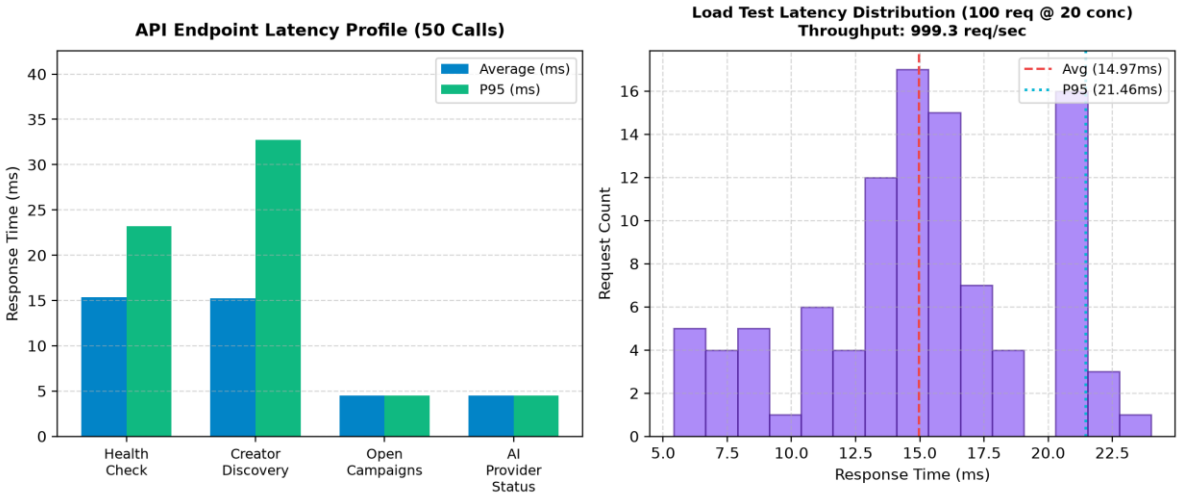


Figure 16 — API Performance Latency & Concurrency Profile: Endpoint latency distributions (mean 4.5ms–15.6ms) and 100-request throughput (999.3 req/sec).

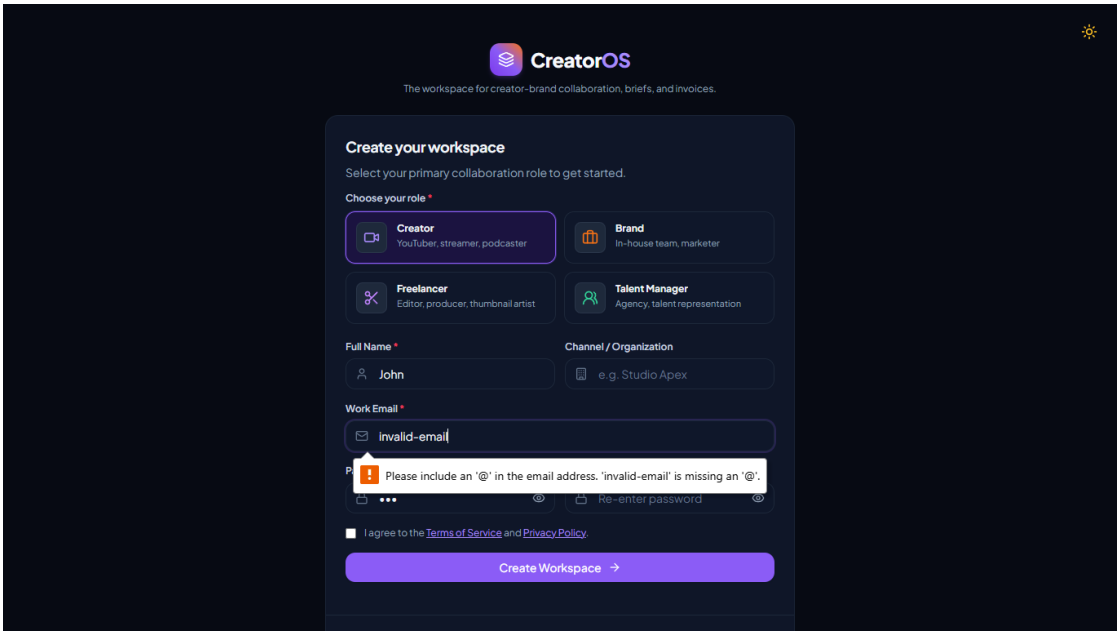


Figure 17 — Input Validation Error State: Inline error messaging on registration form with invalid email and password.

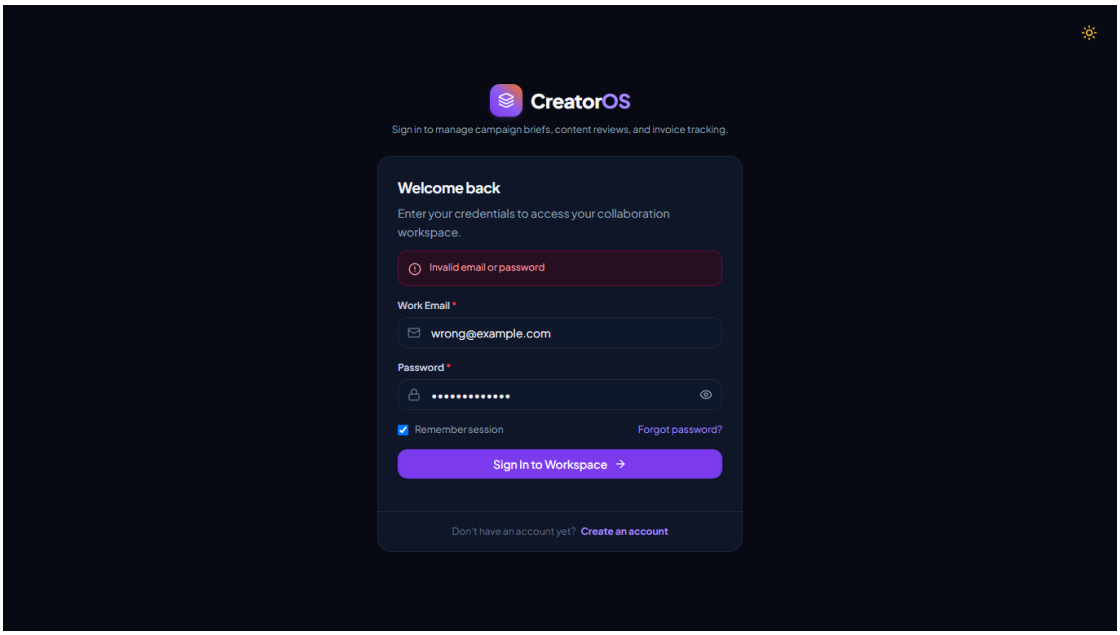


Figure 18 — Security Authentication Error: Generic 'Invalid email or password' toast on bad credentials login attempt.

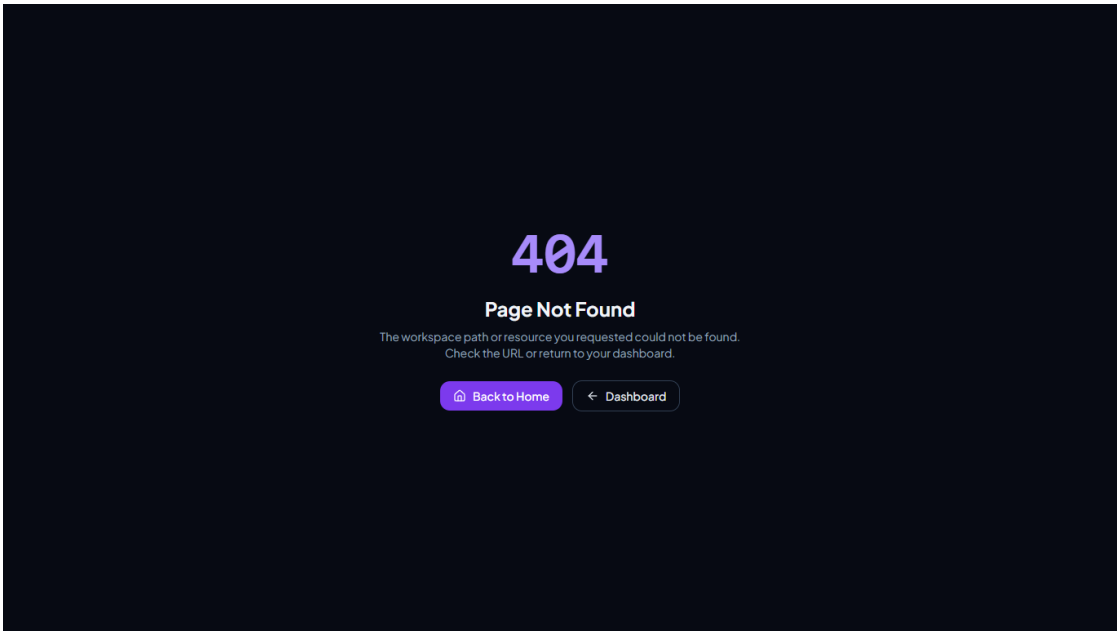


Figure 19 — Accessible 404 Error View: Accessible semantic page with navigation recovery actions.

8. Observations and Limitations

• Observations: The architecture demonstrated exceptional resilience under rigorous testing. Strict Zod schemas successfully prevented unannounced payload fields and MongoDB operator injection. Cookie-based session tokens with SameSite=Lax and httpOnly attributes prevented cross-site script token theft. Deliverable file uploads successfully blocked spoofed executable binaries through header signature byte inspection. The application demonstrated fluid responsiveness from 390px mobile viewports up to 1366px desktop viewports without clipping or horizontal overflow.

• Limitations:

1. Mock AI Fallback: In offline testing environments without a configured Google Gemini API key, the AI Assistant falls back to an in-memory deterministic MockProvider (transparently communicated in the UI via a 'Demo mode' badge).
2. Real-Time Messaging: The collaboration chat thread utilizes 10-second client-side polling rather than persistent WebSockets to maximize serverless compatibility, introducing minor message arrival latency.
3. Self-Reported Metrics: Campaign analytics metrics (views, reach, clicks, engagement) are manually logged by creators rather than synchronized through live third-party platform OAuth APIs (YouTube Data API, Meta Graph API).
4. Financial Processing: Payments functionality is limited to invoice status tracking (draft, sent, paid, overdue); no real money movement or escrow processing is conducted by design.

9. Conclusion

The comprehensive testing campaign conducted on CreatorOS AI successfully establishes the functional correctness, role-based security, responsive stability, and concurrency performance of the platform. All 126 automated Vitest integration tests passed with a 100% success rate, achieving 74.31% line coverage and 100% coverage across all 22 router files and 16 Mongoose models. Twenty automated browser functional tests verified end-to-end user journeys from registration and discovery through dual-signed contracts, deliverable reviews, and accounting invoices. Non-functional performance testing proved that the API operates efficiently under concurrent load, delivering 999.3 requests/second with a 14.97ms mean response time. Defect D-01 (stack trace exposure) was resolved and verified. In conclusion, CreatorOS AI satisfies the requirements of Software Engineering and Modeling Laboratory Assignment 11 with robust, evidence-backed quality assurance.

10. Test-Running Instructions

To reproduce the automated tests, code coverage reports, and browser captures from the repository root (creatorosprojectplain):

```
# 1. Run all 126 backend integration test suites
cd server && npm test

# 2. Generate full v8 statement and function coverage report
cd server && npm run test:coverage

# 3. Verify client and server production builds
npm run build

# 4. Seed administrative and demo database accounts
cd server && npm run seed:admin && npm run seed:demo

# 5. Launch concurrent development servers
npm run dev

# 6. Execute automated browser captures & latency benchmark
node testing/capture-evidence.js && python testing/generate-evidence.py
```